GR22 Regulations

III B.Tech II Semester

Compiler Design Lab

(GR22A3121)

Department of Computer Science and Engineering

GOKARAJU RANGARAJU
INSTITUTE OF ENGINEERING AND TECHNOLOGY
(Autonomous)

Gokaraju Rangaraju Institute of Engineering and Technology (Autonomous)
Department of CSE

COMPILER DESIGN LAB

COURSE CODE: GR22A3121

Marks Int: 30 Extn: 70 **L:0 T:0 P:4 C: 2**

Vision of the institute:

To be among the best of the institutions for engineers and technologists with attitudes, skills and knowledge and to become an epicentre of creative solutions.

Mission of the institute:

To achieve and impart quality education with an emphasis on practical skills and social relevance.

Department of Computer Science and Engineering

Vision

To be a center of global excellence and to emerge as a valuable resource for industry and society

Mission

1) To produce qualified and competent computer professionals with international standards.
2) To foster innovative and application oriented research capabilities of young minds for the progress of society.
3) To inculcate strong ethical values and professional behavior so as to adapt to the emerging changes in the field of computing technology.

Program Educational Objectives(PEOs):

PEO 1: Graduates will be prepared for a successful career in Computer Science discipline and related skills for industry to meet the needs of the nation and leading industries and also to excel in postgraduate programs.

PEO 2: Graduates will continue to learn and apply the acquired knowledge to solve engineering problems and appreciation of the arts, humanities and social sciences.

PEO 3: Graduates will have good and broad scientific and engineering knowledgebase so as to comprehend, analyze, design and create novel products and solutions for real-time applications.

PEO 4: Graduates will understand professional and ethical responsibility, develop leadership, utilize membership opportunities, develop effective communication abilities, teamwork skills, multidisciplinary approach and life-long learning required for a successful professional career.

Program Outcomes:

PO1. Engineering knowledge: Apply the knowledge of mathematics, science, engineering fundamentals, and an engineering specialization to the solution of complex engineering problems.

PO2. Problem analysis: Identify, formulate, review research literature, and analyze complex engineering problems reaching substantiated conclusions using first principles of mathematics, natural sciences, and engineering sciences.

PO3. Design/development of solutions: Design solutions for complex engineering problems and design system components or processes that meet the specified needs with appropriate consideration for the public health and safety, and the cultural, societal, and environmental considerations.

PO4. Conduct investigations of complex problems: Use research-based knowledge and research methods including design of experiments, analysis and interpretation of data, and synthesis of the information to provide valid conclusions.

PO5. Modern tool usage: Create, select, and apply appropriate techniques, resources, and modern engineering and IT tools including prediction and modeling to complex engineering activities with an understanding of the limitations.

PO6. The engineer and society: Apply reasoning informed by the contextual knowledge to assess societal, health, safety, legal and cultural issues and the consequent responsibilities relevant to the professional engineering practice.

PO7. Environment and sustainability: Understand the impact of the professional engineering solutions in societal and environmental contexts, and demonstrate the knowledge of, and need for sustainable development.

PO8. Ethics: Apply ethical principles and commit to professional ethics and responsibilities and norms of the engineering practice.

PO9. Individual and team work: Function effectively as an individual, and as a member or leader in diverse teams, and in multidisciplinary settings.

PO10. Communication: Communicate effectively on complex engineering activities with the engineering community and with society at large, such as, being able to comprehend and write effective reports and design documentation, make effective presentations, and give and receive clear instructions.

PO11. Project management and finance: Demonstrate knowledge and understanding of the engineering and management principles and apply these to one's own work, as a member and leader in a team, to manage projects and in multidisciplinary environments.

PO12. Life-long learning: Recognize the need for, and have the preparation and ability to engage in independent and life-long learning in the broadest context of technological change.

PROGRAM SPECIFIC OUTCOMES (PSOs)

PSO1: To foster industrial focused technical skills in Computer science and Engineering through value added courses and soft skills amalgamate with academic, to create a futuristic equipped professional.

PSO2: Apply the acquired knowledge and skill, to deliver inventive solutions to social, economic and technological challenges, which leads to our nation's growth.

SYLLABUS

1. Design a lexical analyser for given language (ignore redundant spaces, tabs,comments new lines etc..)
2. Write a program to recognize strings under 'a', 'a*b+', 'abb'.
3. Write a program to implement symbol table formation.
4. Write a program to implement predictive parser table.
5. Write a Program to Compute FIRST() and FOLLOW() For the given grammar.
6. Write a Program to Construct operator precedence parser.
7. Write a program to parse a string using Shift Reduce Parser.
8. Write a program to parse a string using LALR parser.
9. Write a program to implement lexical analyzer functionalities using LEX tool.
10. Design a simple arithmetic calculator using LEX .
11. Write a Lex program to count no of characters, words, lines and special characters in a file
12. Write a program to Implement code generation technique.

COURSE OUTCOMES

1. Summarize the phases of compiler using a programming language.
2. Paraphrase the role of lexical analyzer and use of regular expressions.
3. Develop program for implementing parsing techniques.
4. Experiment the LEX and YACC compiler tools used to develop the simple applications.
5. Build a program for code generation and optimization techniques.

INDEX

S. No	Tasks	Page No.
1	Design a lexical analyser for given language (ignore redundant spaces, tabs, comments new lines etc..)	1
2	Write a program to recognize strings under 'a', 'a*b+', 'abb'.	6
3	Write a program to implement symbol table formation.	9
4	Write a program to implement predictive parser table.	12
5	Write a Program To Compute FIRST() and FOLLOW() For the given grammar	16
6	Write a program to Construct operator precedence parser.	23
7	Write a program to parse a string using Shift Reduce Parser.	25
8	Write a program to parse a string using LALR parser.	28
9	Write a program to implement lexical analyzer functionalities using LEX tool.	30
10	Design a simple arithmetic calculator using LEX .	33
11	Write a Lex program to count no of characters, words, lines and special characters in a file	36
12	Write a program to Implement code generation technique.	38

TASK 1

TASK-1: Design a lexical analyzer for given grammar

Aim **: To design a lexical analyzer for given grammar**

Program:

```
#include<stdio.h>
#include<stdlib.h>
#include<string.h>

void main(void)
{
 char *s=(char *)calloc(sizeof(char),20000);
 char *t=(char *)calloc(sizeof(char),20000);
 char
key[32][12]={"auto","break","case","char","const","continue","default","do","double"
,"else","enum","extern","float","for","goto","if","int","long","register","return","short
","signed","sizeof","static","struct","switch","typedef","union","unsigned","void","vol
atile","while"};
 inti,j,k=0,c;
  char f[20],ch;
  FILE *fp;
 //clrscr();
 printf("enter a file: ");
 scanf("%s",f);
 fp=fopen(f,"r");
 if(fp==NULL)
    printf("file cannot be opened");
 else
 {
  while((ch=fgetc(fp))!=EOF)
  {
    *(s+k)=ch;
    k++;
  }
   *(s+k)='\0';
 }
 printf("%s\n",s);
 printf("----------------------------------------\n");
 i=0;
 x:while(*(s+i)!='\0')
```

```
{
 j=0;c=0;
 if(*(s+i)=='#')
 {
   i++;
   while(*(s+i)!='\n')
   {
     printf("%c",*(s+i));
     i++;
   }
   printf("is a pre processor directive\n");
   i++;
 }
 else if(isalpha(*(s+i))||*(s+i)=='_')
 {
   *(t+j)=*(s+i);
  i++;     j++;
  while((*(s+i)=='_'||isdigit(*(s+i))||isalpha(*(s+i)))&&*(s+i)!='\0')
 {
   *(t+j)=*(s+i);
   i++;
   j++;
  }
  *(t+j)='\0';
 for(k=0;k<32;k++)
 if(strcmp(t,key[k])==0)
 {
  c=1;
  break;
  }
  if(c==1)
   printf("%s is a keyword\n",t);
 else
 {
  if(*(s+i)=='(')
  {
     printf("%s is a method\n",t);
     goto x;
  }
  else
  printf("%s is an identifier\n",t);
 }
 }
 else if(*(s+i)==''')
```

```
{
   printf("\"");
   i++;
   while(*(s+i)!="")
   {
     printf("%c",*(s+i));
     i++;
   }
   printf("\" is an argument \n");
   i++;
}
else if((*(s+i)=='/'&&*(s+i+1)=='/')||(*(s+i)=='/'&&*(s+i+1)=='*'))
{
  i++;
  if(*(s+i)=='/')
  {
    i++;
    while(*(s+i)!='\n')
          {
          printf("%c",*(s+i));
            i++;
           }
   }
  else if(*(s+i)=='*')
  {
    i++;
    while(*(s+i)!='*'&&*(s+i+1)!='/')
          {
           // if(*(s+i)=='\n')
        printf("%c",*(s+i));

            i++;
          }
     i=i+2;
   }
          printf("is a comment line\n");
}
else if(*(s+i)=='['||*(s+i)==']'||*(s+i)=='('||*(s+i)==')'||*(s+i)=='{'||*(s+i)=='}')
{
  printf("%c is a special symbol\n",*(s+i));
  i++;
}
else if(isdigit(*(s+i)))
{
```

```
      if(isdigit(*(s+i))||*(s+i)=='.')
      {
      printf("%c",*(s+i));
       i++;
      while(isdigit(*(s+i))||*(s+i)=='.')
      {
              printf("%c",*(s+i));
              i++;
      }
      printf(" is a number\n");
      }
      else if(isalpha(*(s+i)))
     {
          while(isalpha(*(s+i))||*(s+i)=='_')
       {
          printf("%c",*(s+i));
          i++;
       }
          printf(" is an invalid token\n");
      }
    }
   else if(*(s+i)=='='||*(s+i)=='+'||*(s+i)=='-
'||*(s+i)=='*'||*(s+i)=='/'||*(s+i)=='%'||*(s+i)=='<'||*(s+i)=='>'||*(s+i)=='&'||*(s+i)=='|')
   {
     printf("%c is an operator\n",*(s+i));
      i++;
    }
   else if(*(s+i)==';')
   {
      printf("%c is terminator\n",*(s+i));
     i++;
    }
    else
     i++;
  }
  fclose(fp);
 }
```

OUTPUT

```
C:\Users\srini>cd C:\Users\srini\OneDrive\Desktop\CDPrograms

C:\Users\srini\OneDrive\Desktop\CDPrograms>gcc task1.c -o task1

C:\Users\srini\OneDrive\Desktop\CDPrograms>task1.exe
enter a file: hi.c
/* sample c program */
/* input file to LEX */
#include<stdio.h>
void main()
{
int a,b;
printf("Welcome to GRIET");
printf("Welcome to Hyderabad");

}

--------------------------------------------------
 sample c program is a comment line
 input file to LEX is a comment line
include<stdio.h>is a pre processor directive
void is a keyword
main is a method
( is a special symbol
) is a special symbol
{ is a special symbol
int is a keyword
a is an identifier
b is an identifier
; is terminator
```

Experiments:

1) Write a Program to Determine Special Characters in a given Input.

2) Write a Program to Determine identifiers in a given Input.

TASK 2

TASK 2: Write a program to recognize strings under 'a' , 'a*b+' , 'abb'

Aim: **To write a program to recognize strings under 'a' , 'a*b+' , 'abb'**

Program :

```
#include<stdio.h>
#include<stdlib.h>
#include<string.h>
void main()
{
char s[20],c;
int state=0,i=0;
printf("\n Enter a string:");
gets(s);
while(s[i]!='\0')
 {
  switch(state)
  {
   case 0:c=s[i++];
          if(c=='a')
          state=1;
          else if(c=='b')
          state=2;
          else
          state=6;
          break;
   case 1: c=s[i++];
          if(c=='a')
          state=3;
          else if(c=='b')
          state=4;
          else
          state=6;
          break;
   case 2: c=s[i++];
          if(c=='a')
          state=6;
          else if(c=='b')
          state=2;
          else
          state=6;
```

```
            break;
   case 3: c=s[i++];
            if(c=='a')
            state=3;
            else if(c=='b')
            state=2;
            else
            state=6;
            break;
   case 4: c=s[i++];
            if(c=='a')
            state=6;
            else if(c=='b')
            state=5;
            else
            state=6;
            break;
   case 5: c=s[i++];
            if(c=='a')
            state=6;
            else if(c=='b')
            state=2;
            else
            state=6;
            break;
   case 6: printf("\n %s is not recognised.",s);
            exit(0);
    }
  }
 if(state==1)
     printf("\n %s is accepted under rule 'a'",s);
 else if((state==2)||(state==4))
     printf("\n %s is accepted under rule 'a*b+'",s);
 else if(state==5)
      printf("\n %s is accepted under rule 'abb'",s);
}
```

OUTPUT

```
C:\Users\srini>cd C:\Users\srini\OneDrive\Desktop\CDPrograms

C:\Users\srini\OneDrive\Desktop\CDPrograms>gcc task2.c

C:\Users\srini\OneDrive\Desktop\CDPrograms>gcc task2.c -o task2

C:\Users\srini\OneDrive\Desktop\CDPrograms>task2.exe

 Enter a string:abb

 abb is accepted under rule 'abb'
```

Experiments :

1) Write a Program to recognize strings under “ a*b* ”

2) Write a Program to recognize strings under “ (a/b)* ”

TASK 3

TASK 3: Write a program to Implementation symbol table formation.

Aim**: To implementation of symbol table**

Program:

```
#include<stdio.h>
#include<ctype.h>
#include<stdlib.h>
#include<string.h>
#include<math.h>
void main()
{
 int i=0,j=0,x=0,n;
        void *p,*add[5];
        char ch,srch,b[15],d[15],c;
        printf("Expression terminated by $:");
 while((c=getchar())!='$')
 {
        b[i]=c;
         i++;
 }
 n=i-1;
 printf("Given Expression:");
 i=0;
 while(i<=n)
 {
        printf("%c",b[i]);
         i++;
 }
 printf("\n Symbol Table\n");
 printf("Symbol \t addr \t type");
 while(j<=n)
 {
        c=b[j];
  if(isalpha(toascii(c)))
  {
          p=malloc(c);
          add[x]=p;
          d[x]=c;
          printf("\n%c \t %d \t identifier\n",c,p);
```

```
            x++;
            j++;
  }
  else
  {
          ch=c;
            if(ch=='+'||ch=='-'||ch=='*'||ch=='=')
   {
             p=malloc(ch);
             add[x]=p;
             d[x]=ch;
             printf("\n %c \t %d \t operator\n",ch,p);
             x++;
             j++;
   }
}}}
```

OUTPUT:

```
C:\Users\srini\OneDrive\Desktop\CDPrograms>gcc task3.c -o task3


C:\Users\srini\OneDrive\Desktop\CDPrograms>task3.exe
Expression terminated by $:a+b$
Given Expression:a+b
 Symbol Table
Symbol  addr        type
a       14625168    identifier


 +      14625280    operator


b       14625336    identifier
```

Viva Questions :

1) What is Symbol Table?

2) Which symbol table implementation is based on the property of locality of reference?

3) What is Key Symbol Table?

4) Where is Symbol Tables are stored.

5) What are the Types of Symbol Tables?

TASK 4

Task 4: Write a program to implement predictive parser table

Aim: **To write a program to implement predictive parser table**

Program:

```
#include<stdio.h>
#include<string.h>
int n,z=0,m=0,p,i=0,j=0;
char a[3][10],f[10];
void follow(char c);
void first(char c);
int main()
{
        int i,j,z,n1;
        char t[10];
         int x;
        char c;
         printf("enter no of terminals");
         scanf("%d",&n1);
        scanf("%s",t);
        printf("\n enter the no. of productions");
        scanf("%d",&n);
        printf("\n enter the productions (epsilon=e) enter X and Y as alternates for E' and T'");
        for(i=0;i<n;i++)
         scanf("%s",a[i]);
         for(i=0;i<n;i++)
         printf("%s\n",a[i]);
         for(i=0;i<n1;i++)
          printf("\t%c",t[i]);
          printf("\n");
         for(i=0;i<n;i++)
        {
        if((islower(a[i][2]))&&(a[i][2]!='e'))
        {
                x=0;
                  printf("%c\t",a[i][0]);
                 while(a[i][2]!=t[x])
                {
                    printf("\t");
```

```
        x++;
      }
      printf("%c->%s\n",a[i][0],a[i]+2);
}
 if(isupper(a[i][2]))
   {
    m=0;
    first(a[i][2]);
    for(z=0;z<m;z++)
    {
      if(f[z]=='e')
        {
         for(x=0;x<m;x++)
         f[m]='\0';
          m=0;
          follow(a[i][0]);
        }
      }
     printf("%c\t",a[i][0]);
     for(z=0;z<m;z++)
     {
       x=0;
       while(f[z]!=t[x])
       {
        printf("\t");x++;
        }
        printf("%c->%s\t",a[i][0],a[i]+2);
       }
  printf("\n");
 continue;
 }
 if(a[i][2]=='e')
 {
     m=0;
     follow(a[i][0]);
     printf("%c\t",a[i][0]);
     for(z=0;z<m;z++)
    {
       x=0;
       while(f[z]!=t[x])
       {
```

```
                    printf("\t");x++;
                  }
                  printf("%c->e",a[i][0]);
                }
            printf("\n");
            continue;
        }
       }
    return 0;
}
void follow(char c)
{
    int k,l;
    if(a[0][0]==c)f[m++]='$';
    for(k=0;k<n;k++)
    {
      for(l=2;l<strlen(a[k]);l++)
       {
          if(a[k][l]==c)
         {
           if(a[k][l+1]!='\0') first(a[k][l+1]);
           if((a[k][l+1]=='\0') &&(c!=a[k][0]))
           follow(a[k][0]);
        }
      }
    }
}
void first(char c)
{
    int k;
    if(!isupper(c))
    f[m++]=c;
    for(k=0;k<n;k++)
    {
       if(a[k][0]==c)
    {
            if(a[k][2]=='e')
            follow(a[i][0]);
            else if(islower(a[k][2]))
            f[m++]=a[k][2];
            else
```

```
                first(a[k][2]);
        }
        }
}
```

OUTPUT:

```
C:\Users\srini\OneDrive\Desktop\CDPrograms>task4.exe
enter no of terminals4
abcd

 enter the no. of productions4

 enter the productions (epsilon=e) enter X and Y as alternates for E' and T'
A=aB
B=dN
C=bH
D=cL
A=aB
B=dN
C=bH
D=cL
       a      b      c      d
A      A->aB
B                            B->dN
C             C->bH
D                    D->cL
```

Experiments :

1) Write a Program to construct Symbol Table for the given Grammar

   S-->aAd / aB

   A-->b / c

   B--> ccd /ddc

2) Write a Program to construct Symbol Table for the given Grammar

   A-->BC

   B-->cE / d

TASK 5

TASK 5: Write a Program To Compute FIRST() and FOLLOW() For the given grammar.

Aim **: a Program To Compute FIRST() and FOLLOW() For the given grammar.**

Program:

```
// C program to calculate the First and
// Follow sets of a given grammar
#include <ctype.h>
#include <stdio.h>
#include <string.h>

// Functions to calculate Follow
void followfirst(char, int, int);
void follow(char c);

// Function to calculate First
void findfirst(char, int, int);

int count, n = 0;

// Stores the final result
// of the First Sets
char calc_first[10][100];

// Stores the final result
// of the Follow Sets
char calc_follow[10][100];
int m = 0;

// Stores the production rules
char production[10][10];
char f[10], first[10];
int k;
char ck;
int e;

int main(int argc, char** argv)
{
	int jm = 0;
	int km = 0;
	int i, choice;
	char c, ch;
	count = 8;

	// The Input grammar
```

```
strcpy(production[0], "X=TnS");
strcpy(production[1], "X=Rm");
strcpy(production[2], "T=q");
strcpy(production[3], "T=#");
strcpy(production[4], "S=p");
strcpy(production[5], "S=#");
strcpy(production[6], "R=om");
strcpy(production[7], "R=ST");

int kay;
char done[count];
int ptr = -1;

// Initializing the calc_first array
for (k = 0; k < count; k++) {
	for (kay = 0; kay < 100; kay++) {
		calc_first[k][kay] = '!';
	}
}
int point1 = 0, point2, xxx;

for (k = 0; k < count; k++) {
	c = production[k][0];
	point2 = 0;
	xxx = 0;

	// Checking if First of c has
	// already been calculated
	for (kay = 0; kay <= ptr; kay++)
		if (c == done[kay])
			xxx = 1;

	if (xxx == 1)
		continue;

	// Function call
	findfirst(c, 0, 0);
	ptr += 1;

	// Adding c to the calculated list
	done[ptr] = c;
	printf("\n First(%c) = { ", c);
	calc_first[point1][point2++] = c;

	// Printing the First Sets of the grammar
	for (i = 0 + jm; i < n; i++) {
		int lark = 0, chk = 0;

		for (lark = 0; lark < point2; lark++) {
```

```
                        if (first[i] == calc_first[point1][lark]) {
                                chk = 1;
                                break;
                        }
                }
                if (chk == 0) {
                        printf("%c, ", first[i]);
                        calc_first[point1][point2++] = first[i];
                }
        }
        printf("}\n");
        jm = n;
        point1++;
}
printf("\n");
printf("-----------------------------------------------"
        "\n\n");
char donee[count];
ptr = -1;

// Initializing the calc_follow array
for (k = 0; k < count; k++) {
        for (kay = 0; kay < 100; kay++) {
                calc_follow[k][kay] = '!';
        }
}
point1 = 0;
int land = 0;
for (e = 0; e < count; e++) {
        ck = production[e][0];
        point2 = 0;
        xxx = 0;

        // Checking if Follow of ck
        // has already been calculated
        for (kay = 0; kay <= ptr; kay++)
                if (ck == donee[kay])
                        xxx = 1;

        if (xxx == 1)
                continue;
        land += 1;

        // Function call
        follow(ck);
        ptr += 1;

        // Adding ck to the calculated list
        donee[ptr] = ck;
        printf(" Follow(%c) = { ", ck);
```

```
		calc_follow[point1][point2++] = ck;

		// Printing the Follow Sets of the grammar
		for (i = 0 + km; i < m; i++) {
			int lark = 0, chk = 0;
			for (lark = 0; lark < point2; lark++) {
				if (f[i] == calc_follow[point1][lark]) {
					chk = 1;
					break;
				}
			}
			if (chk == 0) {
				printf("%c, ", f[i]);
				calc_follow[point1][point2++] = f[i];
			}
		}
		printf(" }\n\n");
		km = m;
		point1++;
	}
}

void follow(char c)
{
	int i, j;

	// Adding "$" to the follow
	// set of the start symbol
	if (production[0][0] == c) {
		f[m++] = '$';
	}
	for (i = 0; i < 10; i++) {
		for (j = 2; j < 10; j++) {
			if (production[i][j] == c) {
				if (production[i][j + 1] != '\0') {
					// Calculate the first of the next
					// Non-Terminal in the production
					followfirst(production[i][j + 1], i,
								(j + 2));
				}

				if (production[i][j + 1] == '\0'
					&& c != production[i][0]) {
					// Calculate the follow of the
					// Non-Terminal in the L.H.S. of the
					// production
					follow(production[i][0]);
				}
			}
		}
```

```
	}
}

void findfirst(char c, int q1, int q2)
{
	int j;

	// The case where we
	// encounter a Terminal
	if (!(isupper(c))) {
		first[n++] = c;
	}
	for (j = 0; j < count; j++) {
		if (production[j][0] == c) {
			if (production[j][2] == '#') {
				if (production[q1][q2] == '\0')
					first[n++] = '#';
				else if (production[q1][q2] != '\0'
						&& (q1 != 0 || q2 != 0)) {
					// Recursion to calculate First of New
					// Non-Terminal we encounter after
					// epsilon
					findfirst(production[q1][q2], q1,
							(q2 + 1));
				}
				else
					first[n++] = '#';
			}
			else if (!isupper(production[j][2])) {
				first[n++] = production[j][2];
			}
			else {
				// Recursion to calculate First of
				// New Non-Terminal we encounter
				// at the beginning
				findfirst(production[j][2], j, 3);
			}
		}
	}
}

void followfirst(char c, int c1, int c2)
{
	int k;

	// The case where we encounter
	// a Terminal
	if (!(isupper(c)))
		f[m++] = c;
	else {
```

```
                int i = 0, j = 1;
                for (i = 0; i < count; i++) {
                        if (calc_first[i][0] == c)
                                break;
                }

                // Including the First set of the
                // Non-Terminal in the Follow of
                // the original query
                while (calc_first[i][j] != '!') {
                        if (calc_first[i][j] != '#') {
                                f[m++] = calc_first[i][j];
                        }
                        else {
                                if (production[c1][c2] == '\0') {
                                        // Case where we reach the
                                        // end of a production
                                        follow(production[c1][0]);
                                }
                                else {
                                        // Recursion to the next symbol
                                        // in case we encounter a "#"
                                        followfirst(production[c1][c2], c1,
                                                                c2 + 1);
                                }
                        }
                        j++;
                }
        }
}
```

OUTPUT

```
C:\Users\srini\OneDrive\Desktop\CDPrograms>gcc task5.c -o task5

C:\Users\srini\OneDrive\Desktop\CDPrograms>task5.exe

 First(X) = { q, n, o, p, #, }

 First(T) = { q, #, }

 First(S) = { p, #, }

 First(R) = { o, p, q, #, }

---------------------------------------------

 Follow(X) = { $,  }

 Follow(T) = { n, m,  }

 Follow(S) = { $, q, m,  }

 Follow(R) = { m,  }
```

Experiments :

1)Write a Program to compute FIRST() and FOLLOW() for the given Grammar

T-->aAd / aB

C-->b / c

D--> ccd /ddc

2) Write a Program to compute FIRST() and FOLLOW() for the given Grammar

C-->BC

D-->cE / d

TASK 6

TASK 6: Write a program to Construct operator precedence parser

Aim: **To Construct operator precedence parser**

Program:

```
#include <stdio.h>
#include <string.h>

// function f to exit from the loop
// if given condition is not true
void f()
{
        printf("Not operator grammar");
        exit(0);
}

void main()
{
        chargrm[20][20], c;

        // Here using flag variable,
        // considering grammar is not operator grammar
        inti, n, j = 2, flag = 0;

        // taking number of productions from user
        scanf("%d", &n);
        for (i = 0; i< n; i++)
           scanf("%s", grm[i]);

        for (i = 0; i< n; i++) {
           c = grm[i][2];

    while (c != '\0') {

            if (grm[i][3] == '+' || grm[i][3] == '-'|| grm[i][3] == '*' || grm[i][3] == '/')

                    flag = 1;

             else {

                    flag = 0;
                    f();
            }

             if (c == '$')
             {
```

```
                        flag = 0;
            f();
        }

        c = grm[i][++j];
    }
  }

  if (flag == 1)
    printf("Operator grammar");
}
```

OUTPUT

```
C:\Users\srini\OneDrive\Desktop\CDPrograms>task6.exe
Enter no.of Productions in the Grammar
3
A=A+A
B=B*C
C=V+E
Operator grammar
```

Viva Questions :

1) What is Operator Grammar?

2) What are the Applications of Operator Grammar?

3) What are the Limitations of Operator Grammar?

4) What is the Difference Between Operator Grammar and Operator Precedence Grammar?

5) What is the Precedence Function?

TASK 7

Task 7: Write a program to parse a string using Shift Reduce Parsing.

Aim**: To implement Shift Reduce Parsing**

Program

```
#include<stdio.h>
#include<conio.h>
#include<string.h>
inti,j,l,n,top=0,k,p,sl;
char in[20],apro[20],pro[20][20],sta[20],stack[20];
int s[10];
void null();
void out();
void main()
{
        clrscr();
        printf("\n\t\tEnter the total no of production:\t");
        scanf("%d",&n);
        printf("\n\tEnter the production\n");
        for(i=0;i<n;i++)
                {
                scanf("%s",pro[i]);
                s[i]=strlen(pro[i])-3;
                }
        printf("\n\t\tEnter the input string end with $:\t");
        scanf("\n%s",in);
        printf("\n\t\tInput string is:%s",in);
        printf("\n\nStack\tInput\t\tAction\n");
        printf("\n\n--------------------------------------------\n");
        stack[top]='$';
        l=0;
        out();
        while(in[l]!='$')
        null();
        if(stack[top]==pro[0][0] && stack[top+1]=='\0')
        printf("\t\taccept\n");
        else
        printf("error\n");
        getch();
}
```

```
void null()
{
	int p=0,m;
	if(in[l]!='$')
	stack[++top]=in[l++];
	printf("\t\tshift:%c\n",stack[top]);
	out();
	while(1)
		{
		if(stack[top-p]=='$')
		break;
		else
			{
			for(m=top-p,k=0;m<=top;m++,k++)
			sta[k]=stack[m];
			sta[k]='\0';
			sl=strlen(sta);
			for(i=0;i<n;i++)
				{
				//printf("s1=%d\t%d",sl,s[0]);
				if(sl==s[i])
					{
					for(j=3,k=0;pro[i][j]!='\0';j++)
						{
						apro[k]=pro[i][j];
						k++;
						}
					apro[k]='\0';
					if((strcmp(sta,apro))==0)
						{
						top=top-p;
						p=0;
						stack[top]=pro[i][0];
						stack[top+1]='\0';
						printf("\t\t%s\n",pro[i]);
						out();

						}
					}
				}
			}
		p++;
		}
	}
```

```
void out()
        {
        inti;
        for(i=0;i<=top;i++)
        printf("%c",stack[i]);
        printf("\t");
        for(i=l;in[i]!='\0';i++)
        printf("%c",in[i]);
        }
```

OUTPUT:

```
C:\Users\srini\OneDrive\Desktop\CDPrograms>gcc task7.c -o task7

C:\Users\srini\OneDrive\Desktop\CDPrograms>task7.exe

Enter the total no of production:3

Enter the production
E->E+E
E->E*E
E->i

 Enter the input string end with $:     i+i$

               Input string is:i+i$

Stack   Input          Action


-------------------------------------------
$       i+i$           shift:i
$i      +i$            E->i
$E      +i$            shift:+
$E+     i$             shift:i
$E+i    $              E->i
$E+E    $              E->E+E
$E      $              accept
```

Viva Questions :

1) What are the Actions Performed Under Shift- Reduce Parser.

2) What happens During Shift Operation?

3) What happens During Reduce Operation?

4) What happens During Accept Operation?

5) Shift Reduce Parser belongs to Which Parsing Method?

TASK 8

TASK 8: Write a program to parse a string using LALR parser.

Aim **: To write a program to implement LALR parser**

Program:

```
%{
        #include
        #include "y.tab.h"
%}
%%
        [0-9]+ {yylval.dval=atof(yytext);
        return DIGIT;
}
\n|. returnyytext[0];
%%
%{
/*This YACC specification file generates the LALR parser for the program
c
#include
%}
%union
{
        doubledval;
}
        %token DIGIT
        %type expr
        %type term
        %type factor
        %%
        line: expr '\n' {
        printf("%g\n",$1);
}
;
        expr: expr '+' term {$$=$1 + $3 ;}
        | term
        ;
        term: term '*' factor {$$=$1 * $3 ;}
        | factor
        ;
        factor: '(' expr ')' {$$=$2 ;}
        | DIGIT
```

```
        ;
%%

int main()
{
        yyparse();
}
yyerror(char *s)
{
        printf("%s",s);
}
```

OUTPUT:

```
C:\Users\srini\OneDrive\Desktop\CDPrograms>flex task8.l

C:\Users\srini\OneDrive\Desktop\CDPrograms>bison -dy task8.y

C:\Users\srini\OneDrive\Desktop\CDPrograms>gcc lex.yy.c y.tab.c
task8.l: In function 'yylex':
task8.l:12:8: warning: implicit declaration of function 'yyerror' [-Wimplicit-function-declaration]
 .            {ECHO; yyerror("unexpected character");}
       ^~~~~~

C:\Users\srini\OneDrive\Desktop\CDPrograms>a.exe
5+3
1
result is 8
```

Viva Questions:

1) What is LEX?

2) What is yylex()?

3) What is yytext?

4) What is yyparse()?

5) Why we have to include 'y.tab.h' in lex?

TASK 9

TASK 9: Write a program to Implement of Lexical Analyzer functionalities using Lex tool

Aim: **To implement Lexical Analyzer using Lex tool**

```
%{
        int COMMENT=0;
        intcnt=0;
%}
        identifier [a-zA-Z][a-zA-Z0-9]*
%%
#.* { printf("\n%s is a PREPROCESSOR DIRECTIVE",yytext);}
        int |
        float |
        char |
        double |
        while |
        for |
        do |
        if |
        break |
        continue |
        void |
        switch |
        case |
        long |
        struct |
        const |
        typedef |
        return |
        else |
goto {printf("\n\t%s is a KEYWORD",yytext);}
"/*" {COMMENT = 1;}
"*/" {COMMENT = 0; cnt++;}
{identifier}\( {if(!COMMENT)printf("\n\nFUNCTION\n\t%s",yytext);}
\{ {if(!COMMENT) printf("\n BLOCK BEGINS");}
\} {if(!COMMENT) printf("\n BLOCK ENDS");}
{identifier}(\[[0-9]*\])? {if(!COMMENT) printf("\n %s IDENTIFIER",yytext);}
\".*\" {if(!COMMENT) printf("\n\t%s is a STRING",yytext);}
[0-9]+ {if(!COMMENT) printf("\n\t%s is a NUMBER",yytext);}
\)(\;)? {if(!COMMENT) printf("\n\t");ECHO;printf("\n");}
```

```
\( ECHO;
= {if(!COMMENT)printf("\n\t%s is an ASSIGNMENT OPERATOR",yytext);}
\<= |
\>= |
\< |
== |
\> {if(!COMMENT) printf("\n\t%s is a RELATIONAL OPERATOR",yytext);}
%%
int main(intargc,char **argv)
{
if (argc> 1)
{
	FILE *file;
	file = fopen(argv[1],"r");
	if(!file)
	{
		printf("could not open %s \n",argv[1]);
		exit(0);
	}
	yyin = file;
}
	yylex();
	printf("\n\n Total No.Of comments are %d",cnt);
	return 0;
}
intyywrap()
{
	return 1;

}
```

OUTPUT

```
C:\Users\srini\OneDrive\Desktop\CDPrograms\task9>a.exe hi.c



#include<stdio.h> is a PREPROCESSOR DIRECTIVE

        void is a KEYWORD

FUNCTION
        main(
        )


 BLOCK BEGINS

        int is a KEYWORD
 a IDENTIFIER,
 b IDENTIFIER;


FUNCTION
        printf(
        "Welcome to GRIET" is a STRING
        );



 BLOCK ENDS


 Total No.Of comments are 2
```

Viva Questions:

1) What is Auxuliary Definitions?

2) What is Translation Rules?

3) What is Pattern?

4) What is lexeme?

5) What is lex.yy.c?

TASK 10

TASK 10: Design a simple arithmetic calculator using LEX

Aim: **To design a simple arithmetic calculator using LEX**

Program:

```
%{
        int op=0,i;
        floata,b;
%}

dig [0-9]+|([0-9]*)"."([0-9]+)
add "+"
sub "-"
mul "*"
div "/"
pow "^"
ln \n

%%
        {dig} {digi();} /*** digi() is a user defined function ***/
        {add} {op=1;}
        {sub} {op=2;}
        {mul} {op=3;}
        {div} {op=4;}
        {pow} {op=5;}
        {ln} {printf("\n the result :%f\n\n",a);}

%%
digi()
{
        if(op==0)
        a=atof(yytext); /*** atof() is used to convert the ASCII input to float***/
        else
        {
        b=atof(yytext);
        switch(op)
        {
                case 1:a=a+b;
                break;
                case 2:a=a-b;
                break;
```

```
                case 3:a=a*b;
                break;
                case 4:a=a/b;
                break;
                case 5:for(i=a;b>1;b--)
                a=a*i;
                break;
        }
        op=0;
        }
}
main(intargv,char *argc[])
{
yylex();
}
yywrap()
{
return 1;
}
```

OUTPUT:

```
C:\Users\srini\OneDrive\Desktop\CDPrograms\task10>flex task10.l

C:\Users\srini\OneDrive\Desktop\CDPrograms\task10>gcc lex.yy.c
task10.l: In function 'yylex':
task10.l:15:2: warning: implicit declaration of function 'digi' [-Wimplicit-function-declaration]
 {dig} {digi();} /*** digi() is a user defined function ***/
  ^~~~
task10.l: At top level:
task10.l:24:1: warning: return type defaults to 'int' [-Wimplicit-int]
 digi()
 ^~~~
task10.l:48:1: warning: return type defaults to 'int' [-Wimplicit-int]
 main(int argv,char *argc[])
 ^~~~
task10.l:52:1: warning: return type defaults to 'int' [-Wimplicit-int]
 yywrap()
 ^~~~~~

C:\Users\srini\OneDrive\Desktop\CDPrograms\task10>a.exe
5+6

 the result :11.000000
```

Viva Questions:

1. What is the Role of LEX?

2. Give the example for Auxuliary definition?

3. Give the example for Translation Rules?

4. What is Action?

5. What is Parser?

TASK 11

TASK 11: Write a Lex program to count the number of characters, words, lines, and special characters in a file

Aim**: To write a lex program to count the number of characters, words, lines, and special characters in a file**

Program:

```
%{
        #include<stdio.h>
        int lines=0, words=0,s_letters=0,c_letters=0, num=0, spl_char=0,total=0;
%}
%%

        \n { lines++; words++;}
        [\t ' '] words++;
        [A-Z] c_letters++;
        [a-z] s_letters++;
        [0-9] num++;
        . spl_char++;
%%

main(void)
{
        yyin= fopen("myfile.txt","r");
        yylex();
        total=s_letters+c_letters+num+spl_char;
        printf(" This File contains ...");
        printf("\n\t%d lines", lines);
        printf("\n\t%dwords",words);
        printf("\n\t%d digits", num);
        printf("\n\t%d special characters",spl_char);
        printf("\n\tIn total %d characters.\n",total);
}

intyywrap()
{
        return(1);
}
```

OUTPUT:

```
C:\Users\srini\OneDrive\Desktop\CDPrograms\task11>flex task11.l

C:\Users\srini\OneDrive\Desktop\CDPrograms\task11>gcc lex.yy.c
task11.l:14:1: warning: return type defaults to 'int' [-Wimplicit-int]
 main(void)
 ^~~~

C:\Users\srini\OneDrive\Desktop\CDPrograms\task11>a.exe
 This File contains ...
        7 lines
        22words
        12 digits
        7 special characters
        In total 78 characters.
```

Viva Questions :

1) Write the Regular Expression for Identifier?

2) Write the Regular Expression for Constant?

3) Write the Syntax to read a file?

4) Define Pattern, lexeme?

5) What is lex.yy.c?

TASK 12

TASK 12: Write a program to Implement code generation techniques

Aim**: To implement code optimization techniques**

Program:

```
#include<stdio.h>
#include<conio.h>
#include<string.h>
struct regis
{
 char var;
}reg[10];
int noreg=0;
char st[10][10];

int nost;
char* opcode[10]={"ADD","SUB","MUL","DIV"};
char oper[10]={'+','-','*','/'};
char* toopcode(char opert)
{
        int i;
        for(i=0;i<4;i++)
        {
         if(oper[i]==opert)
        {
         return(opcode[i]);
        }
        }
}

int isinregister(char var)
{
        int i;
        for(i=1;i<=noreg;i++)
        {
         if(var==reg[i].var)
```

```
            {
             return(i);
          }
          }
          return(0);
}

void main()
{
          int i,regno2=0,regno1=1,k,j;
          int nost1,n,flag=0;

          clrscr();
          printf("TO GENERATE OPTIMIZED TARGET MACHINE CODE FOR AN
          INTERMEDIATE CODE");
          printf("\nEnter the no. of statements:");
          scanf("%d",&nost);
          nost1=nost;
          printf("Enter the statements:");
          for(i=0;i<nost;i++)
          {
           scanf("%s",st[i]);

          }

           for(k=0;k<nost-1;k++)
          {
             for(j=2;j<5;j++)
                {
                    if(st[k][j]!=st[k+1][j])
                    flag=1;
                 }

         }
printf("If the machine architecture is having the following format \n OPERATIONS
SOURCE  TARGET\n");
printf("ADD var/reg var/reg :--> MOV b,r1 ,variable b contents are moved to register r1\n");
```

```
printf("\n\n\tTstatements    \t\t\t targetcode");
for(i=0;i<nost;i++)
{
 printf("\n\t%s",st[i]);

if((!regno1==isinregister(st[i][2])))
{
        printf("\n\t\t\t\t MOV %c,r%d",st[i][2],++noreg);
        reg[noreg].var=st[i][2];
        regno1=noreg;
}
if((!regno2==isinregister(st[i][4])))
{
        printf("\n\t\t\t\t %s r%d,r%d\n",toopcode(st[i][3]),regno2,regno1);
        reg[regno1].var=st[i][0];
}
else
{
        printf("\n\t\t\t\t %s %c,r%d\n",toopcode(st[i][3]),st[i][4],regno1);
        reg[regno1].var=st[i][0];
}
//if(i==nost-1)
{
printf("\t\t\t\t MOV r%d,%c\n",regno1,st[i][0]);
}

if(flag==0)
 {
 //printf("\t\t\t\t MOV r%d,%c\n",regno1,st[i+1][0]);
 printf("\t%s\t\t\t MOV r%d,%c\n",st[i+1],regno1,st[i+1][0]);
 break;
  }

}
printf("\n CODE GENERATION COMPLETED\n\n");
}
```

OUTPUT:

```
C:\Users\srini\OneDrive\Desktop\CDPrograms>gcc task12.c -o task12

C:\Users\srini\OneDrive\Desktop\CDPrograms>task12.exe
TO GENERATE OPTIMIZED TARGET MACHINE CODE FOR AN INTERMEDIATE CODE
Enter the no. of statements:2
Enter the statements:A=B+C
D=E-F
If the machine architecture is having the following format
 OPERATIONS SOURCE  TARGET
ADD var/reg var/reg :--> MOV b,r1 ,variable b contents are moved to register r1


       Tstatements                 targetcode
       A=B+C
                           MOV B,r1
                           ADD C,r1
                           MOV r1,A

       D=E-F
                           MOV E,r2
                           SUB F,r2
                           MOV r2,D

 CODE GENERATION COMPLETED
```

Viva Questions:

1) What is the function of Code Generation phase?

2) What is Object form?

3) What are the different Types of Object Forms?

4) What is get-reg( ) Function will do?

5) What is Absolute Machine Code?